Binary Search Tree (BST) Operations

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node* create(int x) {
    struct node* n = malloc(sizeof(struct node));
    n->data = x;
    n->left = n->right = NULL;
    return n;
}

struct node* insert(struct node* root, int x) {
    if (root == NULL)
        return create(x);

    if (x < root->data)
        root->left = insert(root->left, x);
    else if (x > root->data)
        root->right = insert(root->right, x);

    return root;
}

int height(struct node* root) {
    if (root == NULL)
        return 0;

    int l = height(root->left);
    int r = height(root->right);
    return 1 + (l > r ? l : r);
}

int minimum(struct node* root) {
    while (root->left != NULL)
        root = root->left;
    return root->data;
}

void swap(struct node* root) {
    if (root == NULL)
        return;

    struct node* temp = root->left;
    root->left = root->right;
    root->right = temp;

    swap(root->left);
    swap(root->right);
}

struct node* search(struct node* root, int x) {
    if (root == NULL || root->data == x)
        return root;

    if (x < root->data)
        return search(root->left, x);
    return search(root->right, x);
}

void inorder(struct node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct node* root = NULL;
```

```
    int n, x, choice;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter values: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        root = insert(root, x);
    }

    do {
        printf("\\n\\n1.Insert Node");
        printf("\\n2.Longest Path");
        printf("\\n3.Minimum Value");
        printf("\\n4.Swap Left and Right");
        printf("\\n5.Search Value");
        printf("\\n6.Display Inorder");
        printf("\\n0.Exit");
        printf("\\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &x);
                root = insert(root, x);
                break;

            case 2:
                printf("Nodes in longest path: %d",
                       height(root));
                break;

            case 3:
                if (root != NULL)
                    printf("Minimum value: %d",
                           minimum(root));
                else
                    printf("Tree is empty");
                break;

            case 4:
                swap(root);
                printf("Left and right pointers swapped");
                break;

            case 5:
                printf("Enter value to search: ");
                scanf("%d", &x);
                if (search(root, x))
                    printf("Value found");
                else
                    printf("Value not found");
                break;

            case 6:
                printf("Inorder: ");
                inorder(root);
                break;

            case 0:
                printf("Exiting...");
                break;

            default:
                printf("Invalid choice");
        }
    } while (choice != 0);

    return 0;
}
```